Message-Level Detection of Support-Team and Recruiter Impersonation on Consumer Platforms: A Layered Detector and Adversarial Benchmark

Sochima Ezema
Independent researcher
sochimaezema00@gmail.com

October 2026 (draft v0.2, detector v0.3.0)

Abstract

Enterprise email security protects employees behind a corporate mail gateway. The people most exposed to employment and support-impersonation scams—students and early-career job seekers—are contacted *inside* consumer platforms, where no gateway sees the message. We present a dependency-free, explainable detector that works at the level of the individual message and its sender, and a public benchmark for comparing such detectors under cheap adversarial transformation. The detector has four layers: (1) deterministic lookalike checks built on Unicode UTS #39 confusable skeletons, mixed-script detection, bounded edit distance, and brand-agnostic detection of a domain hidden inside a link host; (2) a logistic-regression sender scorer over 23 named features trained on a synthetic generator spanning eight brands (holdout F_1 0.987); (3) a message-intent classifier, with a rule baseline and a prompt-injection-hardened LLM wrapper, that receives structured sender facts from the first two layers and whose impersonation verdicts are discounted deterministically only when the sender is verified or unremarkable, nothing sensitive is asked, links stay under the sender's control, and the body does not claim the platform's authority; and (4) a 76-message, brand-parametric benchmark with ten label-preserving adversarial variants and three model-paraphrased corpora, scored on per-layer recall, false-positive rate, injection hijack rate, and latency. On a paraphrased corpus frozen before a single evaluation, all three Layer 3 backends reach recall 1.00 in the fused pipeline, at FPR 0.03 for the rule baseline and Gemma 3 27B and 0.10 for Llama 3.1 8B, against 0.83/0.03, 1.00/0.07 and 1.00/0.47 for the same backends unfused: the sender layers are worth 17 points of recall to the rules and 37 points of FPR to the small model. The 8B model obeyed an appended injection on 6 of 46 attacks, each caught only by a deterministic overrule; the 27B model obeyed none. Three findings fall out: UTS #39 skeletons do not fold Cyrillic U+0442 or Greek U+03BA to ASCII, so skeleton equality alone misses a mixed-script brand lookalike; regex intent cues lose 33 points of recall to a single confusable glyph unless text is confusable-folded first; and negation must be handled at clause level with an exception for conditionals. Code, weights, corpora, and per-message verdict dumps are released under the MIT license.

1 Introduction

A student receives a message from "Handshake Security Team" saying that unusual login activity was detected and that the account will be suspended unless they verify their identity at a link within 24 hours. A job seeker is told they have been selected, with no interview, for a $45-per-hour remote data-entry role and should continue the conversation on Telegram. Both arrive through the platform's own inbox or an address it exposed, not through a corporate mail server with a secure email gateway in front of it. The platform operator is the only party positioned to inspect the message before the recipient does, and usually has no mail-security stack to call.

The U.S. Federal Bureau of Investigation's Internet Crime Complaint Center recorded 20,044 employment-scam complaints and $264,223,271 in employment-scam losses in 2024, up from $70,234,079 in 2023 and $52,204,269 in 2022, a more-than-threefold rise in one year [4]. Existing open-source defences, however, operate at the wrong level for this setting. Domain-permutation tools such as dnstwist [13] and confusable detectors such as `confusable_homoglyphs` [5] judge a registered domain, not a message. Fake-job-posting classifiers [14] judge a public listing, not a direct message to one recipient. LLM phishing benchmarks [12, 15] are email-centric and assume a mail-gateway vantage point with headers and attachments.

Threat model. The victim is a student or early-career job seeker on a consumer career platform. The attacker can send a message with a chosen display name, a chosen sending address or handle, a subject, a body, and links. The attacker may (a) imitate the platform's own support, security, admin, or trust team; (b) imitate a recruiter or employer; (c) ask for credentials or one-time codes; or (d)

redirect a payment. The attacker can cheaply obfuscate the message with confusable glyphs, zero-width characters, digit substitutions, letter spacing, synonyms, lookalike link hosts, or a lookalike sending domain (Section 4 lists ten such transforms). If the defender uses a language model, the attacker can append text that tries to steer the model's output [7, 11]. The defender sees only what the recipient sees: no SPF/DKIM results, no mail headers, no attachment content, and no cross-message history. The defender knows a short list of *protected names*: the platform's domains, brand, and team names.

Contributions.

1. A four-layer, zero-runtime-dependency detector (Section 3) in which every layer returns the reasons for its score, and in which the sender layers hand the intent classifier structured facts and discount its verdict under explicit, auditable conditions.
2. A hardening recipe for using an LLM as an intent classifier on attacker-controlled text (Section 3.3), implementing OWASP LLM01 guidance with an independent prefilter, nonce-delimited untrusted input, schema validation, and fail-closed behaviour.
3. A brand-parametric benchmark with ten adversarial variants, three model-paraphrased corpora, and explicit metrics for injection hijack and per-layer contribution (Section 4), with results for the rule backend and two local LLM backends (Section 5), a stated development/held-out protocol, and an ablation isolating confusable folding and de-obfuscation (Table 4).
4. A candid account of what the benchmark cannot show (Section 6): the development sets are in-sample, the first held-out corpus was consumed by a design fix, the corpus is small and English-only, and only two local LLMs have been measured.

2 Related work

Homograph and lookalike domains. Gabrilovich and Gontmakher [6] introduced the homograph attack; Holgers et al. [8] measured its prevalence in the wild and found it rare but feasible; Hu et al. [9] tested the internationalised-domain-name policies of major browsers against more than 9,000 cases and found weaknesses in every one. UTS #39 [2] is the Unicode Consortium's standard for confusable detection (the *skeleton* function, §4) and restriction levels on script mixing (§5). dnstwist [13] and `confusable_homoglyphs` [5] answer "is this domain a lookalike?" Our Layer 1 reuses the UTS #39 machinery but applies it to every name-like surface of a message (display name, sending domain, each link host) against a protected set, and reports the reason for each match.

Recruitment fraud. Vidros et al. [14] characterised online recruitment fraud and released a public corpus of 17,880 annotated job advertisements, on which most later fake-job classifiers are trained. That line of work classifies the *posting*. The scams in our threat model are one-to-one messages that may never correspond to a posting, and whose tell is a request (fee, equipment purchase, check deposit, off-platform move, personal documents) rather than the listing's wording.

LLM phishing benchmarks and prompt injection. Fraud-R1 [15] evaluates how well LLMs resist multi-round fraud and phishing *inducements* aimed at the model itself; E-PhishGen [12] generates privacy-preserving phishing *email* corpora and shows that detectors trained on existing public sets degrade on new ones. Both are email-centric; our corpus is platform-message-centric and brand-parametric. Greshake et al. [7] showed that content an LLM-integrated application reads can hijack it, which OWASP lists as the top LLM application risk (LLM01) [11]. A message classifier is a textbook instance: the text it classifies is written by the adversary. Section 3.3 describes our mitigations and Section 4 the metric (hijack rate) that measures them.

3 System

The detector is a Python package, `impersonation-detector` v0.3.0 [3], with no runtime dependencies; an LLM backend is optional. It is instantiated with a list of protected names, e.g. `["handshake.com", "Handshake", "Handshake Support"]`, and analyses one `Message` (body, sender display name, sender address, subject, links) into a `Verdict`: a risk in $[0, 1]$, an action in {allow, warn, hold, block}, and a list of human-readable reasons. The seed corpus and all examples here are instantiated with Handshake as the protected brand; every command takes `-p` names and nothing in the code is specific to it.

3.1 Layer 1: deterministic lookalike checks

Layer 1 asks whether a candidate string resembles a protected name and, if so, why. For each candidate (display name, sending domain, every link host in body or links) and each protected name it computes:

- **confusable-skeleton**: equality of case-folded UTS #39 skeletons [2]. The skeleton is NFD normalisation, replacement of each character by its prototype from `confusables.txt`, and NFD again. We casefold before and after, because some prototypes are upper case (`0`→`O`) and a single pre-fold would miss them.
- **edit-distance**$\leq k$: optimal-string-alignment (Damerau–Levenshtein) distance [1, 10] between skeletons, with k scaled to the protected name's length: 0 for names of at most 4 characters, 1 up to 8, else 2, so `a.co` is never a typo of `b.co`.

Table 1: Layer 2 features and learned weights (`scorer_weights.json`, v0.3.0); bias −2.74. Examples: brand affix `handshake-verify.app`; brand in subdomain `handshake.com.evil.io`; embedded foreign domain `orbitallabs.io.docs-share.site`; lure words `auth-check.top`; leet `h4ndshake`. Domain features also score link hosts.

Domain feature	Wt.	Display / local feature	Wt.
is_protected	−1.93	disp. brand_token	+0.81
skeleton_match	+3.47	disp. brand_lookalike	+0.81
close_typo	+3.93	disp. role_word	−0.28
mixed_script	+2.42	disp. claims_brand_off_dom.	+3.02
punycode	+0.72	disp. invisible_chars	+2.87
tld_swap	+3.79	disp. mixed_script	+0.65
brand_affix	+4.57	local brand_token	+3.62
brand_in_subdomain	+0.40	local role_word	+1.01
embeds_foreign_domain	+2.02	local brand_lookalike	0.00
lure_words	+5.12		
leet_match	+1.33		
free_mail	−0.29		
missing	0.00		
invisible_chars	+2.87		

- **mixed-script**: the candidate mixes scripts in a way UTS #39's *Highly Restrictive* profile rejects (Latin with Cyrillic, say, but Latin with Han, Hiragana and Katakana is allowed). Reported only alongside another reason.
- **punycode**: the candidate was an `xn-` label that decoded to a match.

An exact, case-insensitive match to the genuine name is excluded. A fifth, *brand-agnostic* check applies to link hosts only: an interior label that is a public TLD, preceded by at least one label, with the real registrable domain still to its right, means the host hides another organisation's domain inside its own (`orbitallabs.io.docs-share.site` hides `orbitallabs.io`; `mail.google.com` does not match). Any lookalike hit or embedded domain sets a risk floor of 0.85.

3.2 Layer 2: explainable sender scorer

Layer 2 is logistic regression over 23 named binary-or-$[0, 1]$ features of a sender, extracted relative to the protected set (Table 1). The same extractor scores a bare link host using the fourteen domain-level features. Because every feature is relative ("is a brand token embedded in the second-level label?"), weights learned on one set of brands transfer to another, and each prediction returns per-feature contributions so the strongest reasons can be displayed. Two features are brand-agnostic: an embedded foreign domain as above, and a *lure-worded* domain whose second-level label consists entirely of words such as *verify*, *secure*, *portal*, *auth*, *docs*, *hr* (`hr-portal.co`, `auth-check.top`, `secure-portal.app`): an attacker's "own domain" is usually one of these, not `acmerobotics.com`.

Training. A seeded synthetic generator produces labelled senders for a protected set: fifteen positive techniques (homoglyph, typo, TLD swap, affix and prefix domains, the brand's domain or a foreign domain embedded in an attacker host, lure-worded domains, leet, free-mail with brand display name, brand display name on an unrelated attacker domain, punycode, homoglyph display name, invisible characters, brand token in the local part) and eight negative families that share surface features: the genuine brand, other companies with role words in the display name, people on free-mail, university offices and students, and legitimate deep subdomains. We generate 800 examples (half positive) for each of eight protected sets (a career platform, a payments company, a software vendor, an exchange, two job boards, a bank, a file-sharing service) and hold out 20%. Plain gradient descent (L_2 penalty 10^{-4}, learning rate 1.0, 600 epochs) in pure Python takes a few seconds. On the 1,280-example hold-out: precision 1.000, recall 0.973, F_1 0.987, AUC 0.995. The weakest techniques are lure-worded domains (0.786 accuracy), leet domains (0.925) and typo domains (0.95); every other technique scores 1.0. The weights are interpretable: a lure-worded domain is the single strongest signal, a brand token in the local part of a non-brand address (`handshake.support@gmail.com`) the next, and a role word in a display name is on its own slightly *negative*, because it describes "Career Services" at a university as readily as an impostor.

3.3 Layer 3: message intent, with sender facts

Layer 3 gives a message one of five labels (`benign`, `support_impersonation`, `recruiter_scam`, `credential_phish`, `payment_redirect`) with a confidence and the subset of fourteen named signals it observed (urgency, credential request, payment request, off-platform move, personal-data request, too-good-to-be-true, authority claim, link or attachment, deceptive link, threat or consequence, secrecy, injection attempt, invisible characters, homoglyph characters). Two classifiers share this interface, and both receive the same *sender facts*.

Sender facts. Layers 1 and 2 reduce what they found to thirteen named facts that Layer 3 may rely on because the message author did not write them: the sender's domain is the platform's own, institutional (`.edu`, `.gov`, `ac.uk`), free webmail, a lookalike, unrelated, or absent; the display name claims the brand; and the link hosts are none, platform-only, off-platform, all on the sender's own domain, a lookalike, or an embedded foreign domain. The rule classifier reads them directly: a lookalike or embedded link adds the `deceptive_link` signal and can carry a credential-phish label with one other cue, and a platform or institutional sender whose links stay safe and who asks for nothing sensitive is a notice, whatever its tone. The LLM reads them in its prompt with a short platform policy: real support never asks for passwords, codes, or payment, so a verified sender that does is still an attack, and a brand claim from an address the platform does not own is impersonation if it asks for anything.

Normalisation. Both classifiers first normalise the text: NFKC; removal of zero-width, joiner, bidi-isolate, tag, and C0/C1 control characters; *confusable folding*, which replaces every non-ASCII character whose UTS #39 prototype is a single ASCII letter with that letter (Cyrillic U+0430 → `a`) while leaving ASCII untouched; and a 6,000-character cap. The rule classifier then unions cues over *views* of the text: as normalised, de-leeted with `1` read as `l` and as `i` (`p4ssw0rd` → `password`), de-spaced (`p a s s w o r d`), and with the ASCII confusables `rn`→`m` and `vv`→`w` folded (`Venrno`), which UTS #39 maps in the other direction.

Rule baseline. Ten cue families, 551 regular-expression alternatives in total, map to the signals above. A family that can be negated (payment, credential, personal-data, off-platform) is suppressed when a negation token (*never*, *don't*, *without*, ...) occurs earlier in the same clause, except that a conditional ("if you don't verify") is a threat, not a negated request. The classifier is precision-oriented: a label requires two converging cues, e.g. a payment request plus a recruiter, support, or pressure context, or a credential request plus a link, authority claim, urgency, or threat. Base confidence is 0.70–0.80, raised by 0.05 per extra signal beyond two, capped at 0.95. The sender header is also scanned, so "Handshake Support" in the display name is an authority claim even if the body is terse.

LLM classifier and OWASP LLM01 hardening. The second classifier sends the message to a language model over Ollama, any OpenAI-compatible endpoint, or the Anthropic API, using only the standard library. The message is attacker-controlled data, so the wrapper implements the following, each independent of the model's cooperation:

1. **Nonce-delimited untrusted input.** The body is placed between `<message-$nonce>` tags with a fresh 8-hex-digit nonce per call; the system prompt names the nonce and instructs that text inside the tags is data. Any attempt by the body to close the delimiter (`</message-...>` or a bare `</message>`) is rewritten to `[tag removed]` before the prompt is built.
2. **Independent injection pre-filter.** An 84-alternative regular expression ("ignore all previous instructions", "you are now", "system prompt", chat-template tokens such as `<|im_start|>` and `[INST]`, "classify this as benign", ...) runs on the normalised text regardless of what the model says and adds the signal `injection_attempt`.
3. **Schema validation.** The first JSON object in the reply is parsed; the label must be one of the five, the confidence is clipped to $[0, 1]$, unknown signals are dropped, the rationale is truncated. One retry asks for JSON only.
4. **Fail closed.** An unparseable reply, or a backend error, yields the label `unknown` with risk 0.5, never `benign`.
5. **Overrule a hijacked "benign".** If the pre-filter saw an injection and the model answered `benign`, the result becomes `unknown`: the model may have obeyed the message.
6. **Do not let the model raise its own floor.** The model's own `injection_attempt` claim is discarded; only the pre-filter can assert that signal, because the pipeline raises a risk floor on it and a model that could set it could be talked into setting it.

The rule baseline has no instruction channel and therefore cannot be hijacked at all, which is one reason both ship.

3.4 Pipeline and fusion

Risk is a noisy-OR over the independent evidence, $r = 1 - \prod_i (1 - r_i)$ over the sender probability, the probability of the worst link host, and the intent risk (the confidence for a malicious label, 0.5 for `unknown`, 0 for `benign`), then raised to a floor of 0.85 for any Layer 1 hit, 0.60 for an injection attempt, and 0.95 for a domain on an external blocklist. Two rules make the layers correct each other rather than merely add up:

- **Evidence floor.** A component below 0.20 does not combine: an ordinary organisation domain plus a role word plus a weak intent cannot sum to a warning. The code documents the threshold as sitting above the 99th percentile (0.151) of Layer 2's score on 3,200 synthetic benign senders.
- **Fusion discount.** An impersonation verdict from Layer 3 is multiplied by 0.25 when four conditions hold. (i) The sender is a verified institution (platform or institutional domain), an organisation on its own domain that Layer 2 finds unremarkable (no brand claim, score below 0.3, which excludes lure-worded domains), or a webmail peer with no brand claim and no links. (ii) Nothing sensitive is asked for, judged by three vetoes, any one of which blocks the discount: the deterministic cue detector over the de-obfuscated views; the model's label when it is an ask type (credential phish, payment redirect); and, for verified senders only, the model's own ask-type signals, so a platform notice the model reads as asking for a code is held, never waved through. (iii) Links go nowhere the sender does not control: none that are lookalike or embedded, and off-platform only if all are on the sender's own domain. (iv) The body does not claim the platform's authority from a non-platform address.

Actions are thresholds on risk: warn at 0.30, hold at 0.60, block at 0.85, all configurable. A `Middleware` helper wraps a platform's send function: blocked messages are dropped, held ones delivered with the verdict attached for review. Median end-to-end latency on the benchmark is 0.93 ms per message with the rule backend (Table 3); a local LLM adds seconds (Section 5.2).

4 Benchmark design

Seed corpus. The corpus has 76 hand-written messages: 30 benign, 14 support impersonation, 14 recruiter scam, 10

credential phish, 8 payment redirect (46 malicious). It is *brand-parametric*: `{brand}`, `{domain}`, and `{brand_lower}` are filled from the protected set at load time, so the same corpus tests any platform. No example is taken from a real victim report. The benign side is adversarial by design: 26 of 30 benign messages carry a tag naming a surface cue they share with attacks (salary figures, links, deadlines, role words, account and login language, "background check", "verify", "bank details"), including genuine support replies from the protected domain, a genuine password-changed notification, a genuine trust-and-safety notice about a removed fraudulent posting, and a real employer's reminder never to send bank details by email. Three malicious messages (`s06`, `s11`, `c04`) carry a *spoofed From* on the protected domain itself, so the sender gives no evidence and only the request can be judged. Fifteen malicious messages come from free-mail addresses; 21 contain a link, as do 5 benign ones.

Variants. Each seed message is passed through ten deterministic, label-preserving transforms, seeded per (variant, message) so runs are reproducible. Six apply to every message: *clean*; *homoglyph* (one confusable glyph in each cue word and brand mention in the body); *zero_width* (a zero-width space inside each); *leet* (digit substitutions); *spaced* (cue words letter-spaced); and *synonym*. Four make sense only for attacks and are scored on recall alone: *injection*; *link_homoglyph* (one confusable glyph in each link host); *link_subdomain* (the brand's domain embedded as a subdomain of the attacker's host); and *sender_homoglyph* (the sending domain is a confusable of the brand's domain). Body-level transforms act on a fixed list of 30 cue words and on brand mentions, and leave URLs and bare domains untouched, so a benign message's genuine platform link is never turned into a lookalike (an earlier version did); *synonym* applies 23 fixed rewrite rules; *injection* appends one of three payloads (a bracketed "note to automated review systems", a fake `SYSTEM:` line with a target JSON object, and a chat-template `<|im_start|>system` block).

Paraphrased corpora and protocol. The seed corpus is also the development set for the rule classifier's cue lists, so its numbers are in-sample for that backend. For out-of-sample tests, `impersonation-detector paraphrase` has a model rewrite every message with the same purpose, requests, URLs, amounts, and names but different wording, and `bench -only-extra` scores the rewrites alone. Three such corpora exist. The Llama-paraphrased corpus (Llama 3.1 8B; 75 rows, one rewrite dropped as a refusal) and the first Gemma-paraphrased corpus (Gemma 3 27B; 76 rows) were development sets: every miss and false positive they surfaced was traced to a cause and fixed in the detector, the fusion rules, or (twice) the benchmark itself. The first Gemma corpus was intended as held-out; it exposed a design flaw in the fusion gate (Section 5.2) and was thereby consumed. A second Gemma-paraphrased corpus (76 rows) was then generated with a different instruction (`-style`: casual chat where the original was formal and vice versa, sentence order changed), frozen, and evaluated exactly once after all changes. The LLM backends themselves were never tuned.

Configurations and metrics. Five layer configurations are run: L1, L2, L3, L1+L2, and L1+L2+L3; the LLM backends were run on L3 and L1+L2+L3 for the clean, zero-width, and injection variants. A message is *flagged* by L1 if any lookalike matches; by L2 if the sender or any link scores ≥ 0.5; by L3 if the label is malicious or `unknown`; and by a combined configuration if the pipeline action is not `allow`. For every configuration and variant we report precision, recall, F_1, FPR on the 30 benign messages, per-class recall, `unknown` and *hijack* counts, and median latency. Hijack counts, for the injection variant, the malicious messages labelled malicious on the clean text but `benign` with the injection appended: a flipped detection, not a message never caught.

5 Results

5.1 What each layer sees (rule backend, seed corpus)

Table 2 gives recall and FPR for every configuration and variant, and Table 3 per-class recall on the clean corpus, hijack counts, and latency. All numbers are from `benchmarks/benchmark_rule.json` (v0.3.0), generated on an Apple-silicon laptop under Python 3.12.

Layers are complementary. Layer 1 alone catches 4% of attacks, because it knows only confusables of the brand, but 100% of sender-domain homographs. Layer 2 alone catches 59%: every support impersonation and nine of ten credential phishes from the sender line and link hosts, but almost no recruiter scam, since those never mention the brand. The rule Layer 3 alone catches 98% on the clean corpus and on every variant but synonym (96%); its one standing miss (`c09`, a "Google account recovery" phish naming neither the platform nor a cue the rules key on) is caught in the full pipeline by Layer 2. The fused pipeline is at recall 1.00 and FPR 0.00 on all ten variants.

Injection. With the rule backend the hijack count is 0 of 46: the rule path has no instruction channel, and the prefilter recognises all three payloads. The interesting hijack numbers are the LLM backends' (Section 5.2).

Finding 1: UTS #39 skeletons leave gaps a mixed-script check closes. Consider the sending domain `hаndshаke.com`, in which the italicised letters are Cyrillic U+0430, Cyrillic U+04BB, and Greek U+03BA. The prototypes of U+0430 and U+04BB are ASCII `a` and `h`, but the prototype of Greek small kappa is U+0138 LATIN SMALL LETTER KRA and the prototype of Cyrillic small te

Table 2: Recall / false-positive rate by layer configuration and variant, rule backend, v0.3.0. 46 malicious and 30 benign messages. Malicious-only variants report recall alone.

Config	clean	homoglyph	zero_width	leet	spaced	synonym	inject.	link_hg	link_sub	sender_hg
L1	0.04 / 0.00	0.04 / 0.00	0.04 / 0.00	0.04 / 0.00	0.04 / 0.00	0.04 / 0.00	0.04	0.04	0.04	**1.00**
L2	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59	0.59	0.59	1.00
L3	0.98 / 0.00	0.98 / 0.00	0.98 / 0.00	0.98 / 0.00	0.98 / 0.00	0.96 / 0.00	0.98	0.98	0.98	0.98
L1+L2	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59 / 0.00	0.59	0.59	0.59	1.00
L1+L2+L3	**1.00 / 0.00**	1.00 / 0.00	1.00 / 0.00	1.00 / 0.00	1.00 / 0.00	1.00 / 0.00	1.00	1.00	1.00	1.00

Table 3: Per-class recall on the clean corpus (rule backend), injection hijack and unknown counts (of 46), and median latency per message.

Config	cred.	pay.	recr.	supp.	hijack	unk.	ms
L1	0.20	0.00	0.00	0.00	0	0	0.31
L2	0.90	0.38	0.07	1.00	0	0	0.14
L3	0.90	1.00	1.00	1.00	0	0	0.43
L1+L2	0.90	0.38	0.07	1.00	0	0	0.38
L1+L2+L3	1.00	1.00	1.00	1.00	0	0	0.93

Table 4: Ablation of Layer 3 de-obfuscation (rule backend, recall; FPR was 0.00 in every cell). "fold" = confusable folding; "views" = de-leet, de-space and ASCII-confusable views.

L3 config	clean	homogl.	zero_w.	leet	spaced	synonym
full	0.978	0.978	0.978	0.978	0.978	0.957
no fold	0.978	0.652	0.978	0.978	0.978	0.957
no views	0.978	0.957	0.978	0.717	0.696	0.957
neither	0.978	0.630	0.978	0.717	0.696	0.957

(U+0442) is U+1D1B LATIN LETTER SMALL CAPITAL T: Latin, but not ASCII. The skeleton of the candidate is therefore `handsha`⟨U+0138⟩`e.corn`, not `handshake.corn`, and a detector that relies on skeleton equality alone—the natural reading of UTS #39 §4—does not fire. Our Layer 1 still matches, via edit distance 1 between skeletons plus the mixed-script reason (Cyrillic+Greek+Latin), and Layer 2 scores the sender at probability 1.00 from `domain_close_typo`, `domain_mixed_script`, and `display_claims_brand_off_domain`.

Finding 2: regex cues die on one swapped glyph unless the text is folded first. Table 4 re-runs the L3 configuration with confusable folding disabled, with the de-obfuscation views disabled, and with both disabled (`paper/ablation.py`). Without folding, a single confusable glyph in each cue word drops recall from 0.978 (clean) to 0.652, a loss of 33 points; with folding there is no loss. The views move leet recall from 0.717 to 0.978 and letter-spaced recall from 0.696 to 0.978. FPR is 0.00 in every ablated cell. The lesson generalises: any cue- or token-based classifier sees a different string after a confusable swap, so normalisation belongs in front of the classifier.

Finding 3: negation is clause-level and has a conditional exception. Benign message `b26` is a real employer's onboarding note: HR will send a secure link from the payroll provider to enter direct-deposit details, and "Never send bank details by email." The phrase *bank details* is a payment-request cue; the message also contains a link word and a role word, which is enough context for the rule classifier to label it `payment_redirect` if the cue fires. With the negation guard it is `benign` with no signals; deleting the word *never* yields `payment_redirect`. The guard's first version also silenced "your privileges will be revoked if you don't verify your account", a support impersonation all three backends had labelled correctly; a conditional is a threat, not a negated request.

5.2 Three backends, paraphrased corpora, and a frozen held-out set

Table 5 gives the full pipeline on the development sets for the three Layer 3 backends, and Table 6 the single evaluation on the frozen held-out corpus, with the unfused Layer 3 alongside. Per-message verdicts are in the `dump` files named in the repository.

What the fusion is worth. The contrast between the two columns of Table 6 is the clearest statement of what Layers 1–2 contribute to Layer 3. The rule classifier alone recovers 83% of the rewritten attacks (eight misses, all support impersonations and credential phishes whose wording left the cue lists); the sender facts and link checks bring it to 100% with no new false positive. The 8B model alone flags 14 of 30 benign messages; the fusion discount removes eleven of them without costing a single attack. The 27B model alone flags two (`b17`, `b23`); with the sender facts in its prompt it clears both, but reads the `b08` rewrite as a scam, so it keeps one. On the development sets the pattern is the same and the zeros are the product of iterating, so they are not independent evidence.

The residual false positives are the hard benign cases. The one message all three backends hold on the frozen set, `b08`, is a casual recruiter rewrite: "You'll earn $30/hour working about 40 hours a week, and we provide the laptop." The too-good-to-be-true cue ("earn $") fires, which vetoes the discount, and both models also read it as a scam. Llama additionally holds the platform's password-changed notice (`b17`) and a financial-aid office's document

Table 5: Full pipeline (L1+L2+L3 with fusion), recall / FPR, on the development sets, by Layer 3 backend. The LLMs were run on the clean, zero-width and injection variants of the seed corpus. "Comply" is how many of the 46 injected attacks the model itself labelled `benign` before the overrule; every one was overruled to `unknown`, so the hijack count is 0 throughout.

Backend	seed, clean	seed, zero_width	seed, injection	Llama-paraphrased (75)	Gemma-paraphrased #1 (76)	comply
rule (no model)	**1.00 / 0.00**	1.00 / 0.00	1.00	**1.00 / 0.00**	**1.00 / 0.00**	cannot
Llama 3.1 8B	1.00 / 0.03	1.00 / 0.03	1.00	1.00 / 0.03	1.00 / 0.07	6 of 46
Gemma 3 27B	**1.00 / 0.00**	1.00 / 0.00	1.00	**1.00 / 0.00**	**1.00 / 0.00**	0 of 46

Table 6: Frozen held-out corpus (Gemma-paraphrased #2, 76 messages), evaluated once. Unfused Layer 3 alone versus the fused pipeline, recall / FPR, and the false positives.

Backend	L3 alone	full pipeline	false positives
rule	0.83 / 0.03	**1.00** / 0.03	b08
Llama 3.1 8B	1.00 / 0.47	**1.00** / 0.10	b08, b17, b23
Gemma 3 27B	1.00 / 0.07	**1.00** / **0.03**	b08

request (`b23`) because it reports a credential ask on both; by rule (ii) of Section 3.4 a verified sender whose message the model reads as asking for credentials is held for review, never waved through. That is the designed outcome for a weak model; Gemma, given the same facts, labels every genuine platform notice benign on its own. The spoofed-From attacks (`s06`, `s11`, `c04`) stay caught because each has a sensitive ask or a foreign link.

Two design lessons came from the consumed held-out set, not from the development sets. The first version of the fusion gate let the deterministic cues alone decide whether a sensitive ask was present. A cue list cannot keep up with a paraphrasing model: six messages that both LLMs had labelled correctly ("respond with the six-digit code", "purchase the initial kit") were discounted because the regexes did not recognise the wording. A regex-only veto of a correct model verdict is the wrong shape; the model's own ask-type label now vetoes the discount too. The second lesson is the lure-domain feature: the gate's "organisation on its own domain" clause needs Layer 2 to tell `acmerobotics.com` from `secure-portal.app`, and it can.

Small models invent evidence; the pre-filter earns its place. Llama 3.1 8B's rationales on its unfused false positives cite cues absent from the text (the password-changed notice "requests the user to reset their password"), at confidences of 0.8–0.9 indistinguishable from its real detections; Gemma 3 27B's cited cues are real. Under the injection variant Llama complied with the appended instruction on 6 of 46 attacks when run alone and 4 of 46 with sender facts in the prompt; every one was overruled to `unknown` by the deterministic pre-filter, which is why the hijack count is 0. Gemma complied 0 times. The pre-filter still runs for either, because the guarantee has to hold for whichever model is plugged in.

Cost. LLM latency in the tables comes from the response cache and is not meaningful; uncached, the repository records 2–6 s per message for the 8B model and 8–12 s for the 27B model on an Apple-silicon laptop, against 0.93 ms for the rule pipeline. The backends sit close together on the frozen set, so the choice is cost against the residual benign notices a small model insists are phishing; hosted models run through the same harness.

6 Limitations

Development-set numbers are not evidence; the first held-out set was consumed. The seed corpus, its variants, and the two paraphrased development corpora were all used to iterate on the cue lists and the fusion gate, so their zeros measure the state reached by iterating, not generalisation; the first Gemma-paraphrased corpus was intended as held-out and became a development set when it exposed the gate flaw. The second was frozen and evaluated once, and Table 6 is the number to quote; with 76 messages, one message is 2.2 points of recall and 3.3 points of FPR. Two further caveats: each paraphrased corpus was produced by one of the models then evaluated on it, so an LLM's recall on its own paraphrases is not independent evidence; and a paraphrase preserves the request, so it tests wording, not novel scam scripts. The two LLMs measured are local models at temperature 0, the larger on three variants only; their false-positive rates and injection compliance are not properties of LLM classifiers in general.

The corpus is small and English-only. The corpus is hand-written so that every example is defensible and no victim's message is reproduced, which bounds its size. A platform deploying this should expect to re-run the benchmark on its own labelled traffic; contributions of labelled real-world messages, with consent, are the obvious next step.

Layer 2 is trained on a generator. The generator encodes the impersonation techniques we know; a novel one needs a new named feature, which is why the feature list is explicit and the trainer is 150 lines. The 0.987 holdout F_1 should not be expected to transfer to wild senders.

Protected names are a list, and homonyms exist. A legitimately named business whose name contains the

brand ("Firm Handshake Consulting") scores as a lookalike; the right response is a product decision (warn and show the reason), not a detector change.

Beyond the message. The package also contains two modules outside this paper's scope that apply the same recipe to the domain an attacker registers (a lookalike-domain watch with DNS/MX/RDAP evidence, Layer 2 scoring, and a blocklist fed back to the message detector) and to the account they take over (with a cross-account shared-IP signal). A live scan of `handshake.com` and `joinhandshake.com` on 2026-10-13 (`benchmarks/watch_handshake.json`) returned 72 lookalike hits, 38 with MX records, mostly TLD swaps and single-character omissions: capable of abuse, not evidence of it.

7 Reproducibility

All code, weights, corpora, dumps, and reports are public [3]. The following reproduces every number in this paper except the IC3 figure:

```
cd impersonation-detector    # after git clone; fresh venv
pip install -e ".[dev]"
pytest -q                                # 80 tests
impersonation-detector bench -p handshake.com \
    -p Handshake -p "Handshake Support" --out benchmarks/
python paper/ablation.py                 # ablation table
impersonation-detector bench ... --backend ollama \
    --model llama3.1:8b          # Tables 5-6, LLM rows
impersonation-detector paraphrase ... --backend ollama \
    --model gemma3:27b --style "..." --out heldout2.jsonl
impersonation-detector bench ... --dump dump.jsonl \
    --only-extra heldout2.jsonl
```

The LLM runs need a local Ollama; per-message response caches (`benchmarks/cache_*.jsonl`) preserve the exact responses behind Tables 5 and 6, and the three paraphrased corpora are committed. Layer 2 retraining (`python -m impersonation_detector.train`) is deterministic given `-seed`; variants are seeded per message. Unicode data files are redistributed under the Unicode License.

8 Ethics

No real victim data was used. Every message in the seed corpus was written by the author; senders, companies, and domains are fictional, and the only real organisation named is the protected brand used as the corpus parameter. The adversarial variants and injection payloads are the kind an attacker can already produce in seconds and are published so defenders can measure against them [12, 15]. The detector is explainable by design so that a platform can show a user or reviewer the reason for a hold rather than silently dropping a legitimate recruiter's message. The author is an engineer at the platform used as the example brand; this work was done independently, uses no platform data, and does not describe that platform's production defences.

References

[1] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3): 171–176, 1964. doi: 10.1145/363958.363994.

[2] Mark Davis and Michel Suignard. Unicode technical standard #39: Unicode security mechanisms. Unicode Consortium, `https://www.unicode.org/reports/tr39/`, 2026. Data files `confusables.txt` and `Scripts.txt` as of Unicode 18.0.0 (2026).

[3] Sochima Ezema. impersonation-detector: layered detection of lookalike senders and support-team / recruiter impersonation for messaging platforms. `https://github.com/sochimaezema-hs/impersonation-detector`, 2026. Version 0.2.0, MIT license.

[4] Federal Bureau of Investigation, Internet Crime Complaint Center. 2024 Internet Crime Report. Technical report, FBI, 2025. URL `https://www.ic3.gov/AnnualReport/Reports/2024_IC3Report.pdf`.

[5] Victor Felder. confusable_homoglyphs: Detect confusable usage of unicode homoglyphs. `https://github.com/vhf/confusable_homoglyphs`, 2016. Software.

[6] Evgeniy Gabrilovich and Alex Gontmakher. The homograph attack. *Communications of the ACM*, 45(2):128, 2002. doi: 10.1145/503124.503156.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90. ACM, 2023. arXiv:2302.12173.

[8] Tobias Holgers, David E. Watson, and Steven D. Gribble. Cutting through the confusion: A measurement study of homograph attacks. In *Proceedings of the 2006 USENIX Annual Technical Conference (USENIX ATC '06)*, pages 261–266, Boston, MA, 2006. USENIX Association.

[9] Hang Hu, Steve T. K. Jan, Yang Wang, and Gang Wang. Assessing browser-level defense against IDN-based phishing. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. USENIX Association, 2021. ISBN 978-1-939133-24-3.

[10] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8): 707–710, 1966.

[11] OWASP GenAI Security Project. LLM01:2025 Prompt Injection. OWASP Top 10 for LLM Applications 2025, `https://genai.owasp.org/llmrisk/llm01-prompt-injection/`, 2025.

[12] Luca Pajola, Eugenio Caripoti, Stefan Banzer, Simeone Pizzi, Mauro Conti, and Giovanni Apruzzese. E-PhishGen: Unlocking novel research in phishing email detection. In *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security (AISec '25)*, pages 64–76. ACM, 2025. doi: 10.1145/3733799.3762967.

[13] Marcin Ulikowski. dnstwist: Domain name permutation engine for detecting homograph phishing attacks, typo squatting, and brand impersonation. `https://github.com/elceef/dnstwist`, 2015. Software.

[14] Sokratis Vidros, Constantinos Kolias, Georgios Kambourakis, and Leman Akoglu. Automatic detection of online recruitment frauds: Characteristics, methods, and a public dataset. *Future Internet*, 9(1):6, 2017. doi: 10.3390/fi9010006.

[15] Shu Yang, Shenzhe Zhu, Zeyu Wu, Keyu Wang, Junchi Yao, Junchao Wu, Lijie Hu, Mengdi Li, Derek F. Wong, and Di Wang. Fraud-R1: A multi-round benchmark for assessing the robustness of LLM against augmented fraud and phishing inducements. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4374–4420, 2025. doi: 10.18653/v1/2025.findings-acl.226.